

Q36. Application: Search Engine Indexing

Analysis of Search Engine Indexing Using Master Theorem

Problem Statement

A search engine indexing algorithm is defined by the recurrence relation:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Apply the **Master Theorem** to determine the time complexity. Clearly explain the selected case and analyze how indexing time grows as the number of web pages increases.

1. Understanding the Problem Context

A search engine indexes millions of web pages to make searching fast and efficient. The indexing algorithm divides the collection of web pages into two equal parts and recursively processes each part. After processing, additional operations such as sorting, ranking, and merging indexed data require n^2 time.

The recurrence relation representing the indexing algorithm is:

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

The objective is to determine the algorithm's time complexity using the **Master Theorem**.

2. Application of Master Theorem

The standard form of the Master Theorem is:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Identify the Parameters

- $a = 2$ (Number of recursive calls)
- $b = 2$ (Input size divided into two equal parts)
- $f(n) = n^2$

Now calculate:

$$\begin{aligned} n^{\log_b a} \\ &= n^{\log_2 2} \\ &= n \end{aligned}$$

Thus,

$$n^{\log_b a} = n$$

Compare:

$$f(n) = n^2$$

Since

$$n^2 = \Omega(n^{1+\varepsilon})$$

where

$$\varepsilon = 1,$$

the function $f(n)$ grows polynomially faster than $n^{\log_b a}$.

3. Selection of Master Theorem Case

The three cases of the Master Theorem are:

- **Case 1:** $f(n)$ is asymptotically smaller.

- **Case 2:** $f(n)$ is asymptotically equal.
- **Case 3:** $f(n)$ is asymptotically larger.

Since

$$f(n) = n^2$$

grows faster than

$$n^{\log_b a} = n,$$

Master Theorem Case 3 is applicable.

Regularity Condition

$$af\left(\frac{n}{b}\right) = 2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2}$$

Since

$$\frac{n^2}{2} \leq cn^2$$

for some constant $c < 1$, the regularity condition is satisfied.

Therefore,

$$\boxed{T(n) = \Theta(n^2)}$$

4. Time Complexity

Using **Master Theorem Case 3**,

$$\boxed{T(n) = \Theta(n^2)}$$

Hence, the search engine indexing algorithm has **Quadratic Time Complexity**.

5. Performance Analysis

As the number of web pages increases, the indexing time increases quadratically.

For example:

Number of Web Pages (n)	Approximate Operations (n ²)
100	10,000
500	250,000
1,000	1,000,000
2,000	4,000,000

This means:

- Small collections of web pages can be indexed efficiently.
 - Large-scale search engines with millions of pages require significantly more computation.
 - If the number of web pages doubles, the indexing operations become approximately four times larger.
 - Therefore, search engines often use distributed computing and optimization techniques to improve indexing performance.
-

6. Interpretation Using Asymptotic Notation

The asymptotic complexity is

$$\Theta(n^2)$$

This indicates that the algorithm has a **quadratic growth rate**.

- **Big-O:** $O(n^2)$ – Upper bound
- **Big-Omega:** $\Omega(n^2)$ – Lower bound
- **Big-Theta:** $\Theta(n^2)$ – Tight bound

Thus, the indexing algorithm consistently requires quadratic time for large input sizes.

7. Conclusion

The search engine indexing algorithm follows the recurrence relation

$$T(n) = 2T(n/2) + n^2.$$

Using the **Master Theorem**:

- $a = 2$
- $b = 2$
- $f(n) = n^2$
- $n^{\log_b a} = n$

Since n^2 grows faster than n , **Master Theorem Case 3** applies. The regularity condition is satisfied, resulting in the final time complexity:

$$\boxed{T(n) = \Theta(n^2)}$$

The indexing time increases quadratically as the number of web pages grows. Therefore, while the algorithm performs well for smaller datasets, indexing very large collections of web pages requires additional optimization techniques to maintain efficient performance.

SOURCE CODE:

```
#include <stdio.h>
```

```
int T(int n)
```

```
{
```

```
    if (n <= 1)
```

```
        return 1;
```

```
    return 2 * T(n / 2) + (n * n);
```

```
}
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter the number of web pages: ");
```

```
    scanf("%d", &n);
```

```
    printf("Indexing Result T(%d) = %d\n", n, T(n));
```

```
    scanf("%d", &n);
```

```
    printf("Indexing Result T(%d) = %d\n", n, T(n));
```

```
    printf("Time Complexity using Master Theorem: Theta(n^2)\n");
```

```
    return 0;
```

```
}]
```

mastertheorem.c

SearchEngineIndexing.cpp